

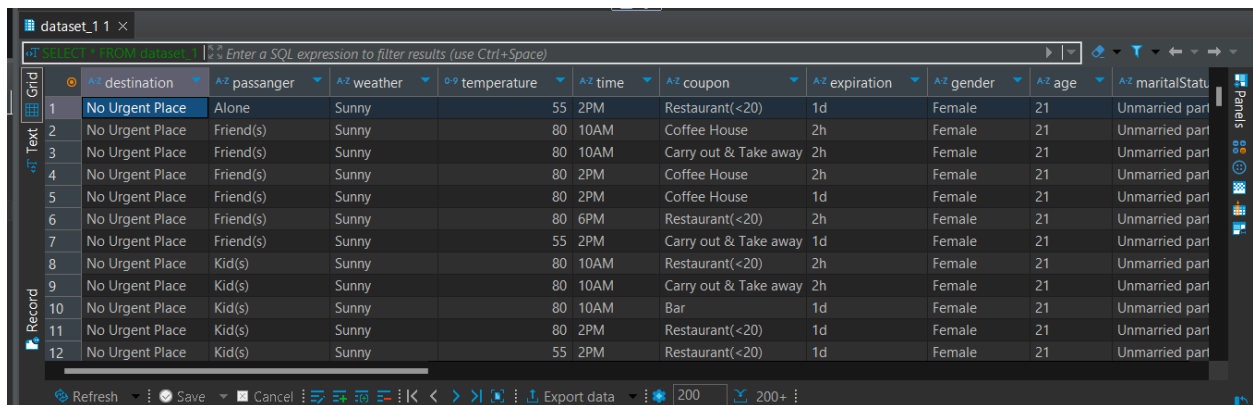
Data Analysis in SQL Vs Python

Author: www.linkedin.com/in/shabanaalina14

- Importing Pandas for data manipulation in Python.
- Using the DBeaver Community version, I used SQLite for database management.

1. SQL:

SELECT * FROM dataset_1 Author: www.linkedin.com/in/shabanaalina14



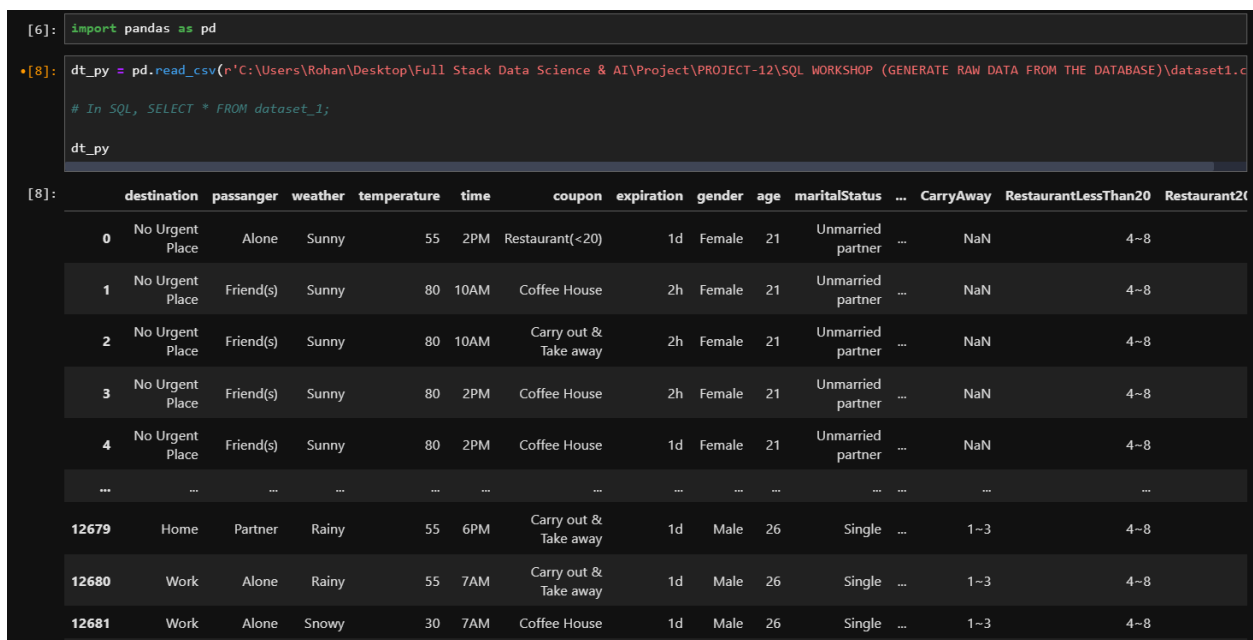
The screenshot shows the DBeaver SQL editor with a query result for 'dataset_1'. The query is 'SELECT * FROM dataset_1'. The result is displayed in a table with 12 rows and 11 columns. The columns are: destination, passenger, weather, temperature, time, coupon, expiration, gender, age, and maritalStatus. The data shows various locations like 'No Urgent Place', 'Home', 'Work', and different weather conditions like 'Sunny', 'Rainy', and 'Snowy'.

	destination	passenger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus
1	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried part
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried part
3	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried part
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried part
5	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried part
6	No Urgent Place	Friend(s)	Sunny	80	6PM	Restaurant(<20)	2h	Female	21	Unmarried part
7	No Urgent Place	Friend(s)	Sunny	55	2PM	Carry out & Take away	1d	Female	21	Unmarried part
8	No Urgent Place	Kid(s)	Sunny	80	10AM	Restaurant(<20)	2h	Female	21	Unmarried part
9	No Urgent Place	Kid(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried part
10	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d	Female	21	Unmarried part
11	No Urgent Place	Kid(s)	Sunny	80	2PM	Restaurant(<20)	1d	Female	21	Unmarried part
12	No Urgent Place	Kid(s)	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried part

Python:

```
dt_py = pd.read_csv(r'C:___\dataset_1.csv')
```

dt_py



The screenshot shows a Jupyter Notebook with two cells. The first cell contains the code to import pandas and read the CSV file. The second cell shows the resulting DataFrame, which has 12 rows and 11 columns. The columns are: destination, passenger, weather, temperature, time, coupon, expiration, gender, age, maritalStatus, and ... (which is a truncated column). The data shows various locations like 'No Urgent Place', 'Home', 'Work', and different weather conditions like 'Sunny', 'Rainy', and 'Snowy'.

```
[6]: import pandas as pd

[8]: dt_py = pd.read_csv(r'C:___\dataset_1.csv')
# In SQL, SELECT * FROM dataset_1;
dt_py
```

	destination	passenger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaurant20
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4-8	
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4-8	
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried partner	...	NaN	4-8	
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4-8	
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried partner	...	NaN	4-8	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
12679	Home	Partner	Rainy	55	6PM	Carry out & Take away	1d	Male	26	Single	...	1-3	4-8	
12680	Work	Alone	Rainy	55	7AM	Carry out & Take away	1d	Male	26	Single	...	1-3	4-8	
12681	Work	Alone	Snowy	30	7AM	Coffee House	1d	Male	26	Single	...	1-3	4-8	

2. SQL:

SELECT gender, income FROM dataset_1

dataset_1 11 x

SQL: SELECT gender, income FROM dataset_1

Enter a SQL expression to filter results (use Ctrl+Space)

Grid

1 Female \$37500 - \$49999

2 Unique key: No unique key was found. Data modification is not possible.

3 Female \$37500 - \$49999

4 Female \$37500 - \$49999

5 Female \$37500 - \$49999

6 Female \$37500 - \$49999

7 Female \$37500 - \$49999

8 Female \$37500 - \$49999

9 Female \$37500 - \$49999

10 Female \$37500 - \$49999

11 Female \$37500 - \$49999

12 Female \$37500 - \$49999

Author: www.linkedin.com/in/shabanaalina14

Refresh Save Cancel

200 200+

200 row(s) fetched - 0.001s (0.001s fetch), on 2024-09-02 at 09:59:07

Python:

dt_py[['gender','income']]

[10]: # In SQL, SELECT gender, income FROM dataset_1

dt_py[['gender','income']]

[10]:

gender income

0 Female 37500-49999

1 Female 37500-49999

2 Female 37500-49999

3 Female 37500-49999

4 Female 37500-49999

...

12679 Male 75000-87499

12680 Male 75000-87499

12681 Male 75000-87499

12682 Male 75000-87499

12683 Male 75000-87499

12684 rows x 2 columns

3. SQL:

SELECT * FROM dataset_1 LIMIT 5

dataset_1 11 x

SQL: SELECT * FROM dataset_1 LIMIT 5

Enter a SQL expression to filter results (use Ctrl+Space)

Grid

1 No Urgent Place Alone Sunny 55 2PM Restaurant(<20) 1d Female 21 Unmarried partner

2 No Urgent Place Friend(s) Sunny 80 10AM Coffee House 2h Female 21 Unmarried partner

3 No Urgent Place Friend(s) Sunny 80 10AM Carry out & Take away 2h Female 21 Unmarried partner

4 No Urgent Place Friend(s) Sunny 80 2PM Coffee House 2h Female 21 Unmarried partner

5 No Urgent Place Friend(s) Sunny 80 2PM Coffee House 1d Female 21 Unmarried partner

Refresh Save Cancel

200 5

5 row(s) fetched - 0.001s, on 2024-09-02 at 10:03:14

Python:

```
dt_py.head(5)
```

```
[12]: # In SQL, SELECT * FROM dataset_1 LIMIT 5
dt_py.head(5)
```

	destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaurant20To50
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4-8	1-3
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4-8	1-3
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried partner	...	NaN	4-8	1-3
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4-8	1-3
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried partner	...	NaN	4-8	1-3

5 rows × 27 columns

4. SQL:

```
SELECT DISTINCT weather FROM dataset_1
```

dataset_11 x

SQL: SELECT DISTINCT weather FROM dataset_1

Grid	1	2	3
Text	Sunny	Rainy	Snowy
Record			

Refresh Save Cancel Filter Sort Export data 200 3 3 row(s) fetched - 0.011s (0.008s fetch), on 2024-09-02 at 10:05:33

Python:

```
dt_py['weather'].unique()
```

```
[14]: # In SQL, SELECT DISTINCT weather FROM dataset_1
dt_py['weather'].unique()

[14]: array(['Sunny', 'Rainy', 'Snowy'], dtype=object)
```

5. SQL:

SELECT *FROM dataset_1 ORDER BY age

	destination	passenger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus
1	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried part
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried part
3	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried part
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried part
5	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried part
6	No Urgent Place	Friend(s)	Sunny	80	6PM	Restaurant(<20)	2h	Female	21	Unmarried part
7	No Urgent Place	Friend(s)	Sunny	55	2PM	Carry out & Take away	1d	Female	21	Unmarried part
8	No Urgent Place	Kid(s)	Sunny	80	10AM	Restaurant(<20)	2h	Female	21	Unmarried part
9	No Urgent Place	Kid(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried part
10	No Urgent Place	Kid(s)	Sunny	80	10AM	Bar	1d	Female	21	Unmarried part
11	No Urgent Place	Kid(s)	Sunny	80	2PM	Restaurant(<20)	1d	Female	21	Unmarried part

Python:

dt_py.sort_values('age')

```
[18]: # In SQL, SELECT *FROM dataset_1 ORDER BY age
dt_py.sort_values('age')
```

	destination	passenger	weather	temperature	time	coupon	expiration	gender	age	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaura
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4~8	
7732	No Urgent Place	Friend(s)	Sunny	80	6PM	Coffee House	1d	Male	21	Unmarried partner	...	1~3	4~8	
7731	No Urgent Place	Friend(s)	Sunny	80	2PM	Restaurant(20-50)	2h	Male	21	Unmarried partner	...	1~3	4~8	
7730	No Urgent Place	Alone	Sunny	55	6PM	Coffee House	2h	Male	21	Unmarried partner	...	1~3	4~8	
7729	No Urgent Place	Alone	Sunny	80	10AM	Coffee House	1d	Male	21	Unmarried partner	...	1~3	4~8	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
11530	Home	Alone	Rainy	55	10PM	Coffee House	2h	Male	below21	Single	...	gt8	gt8	
11529	Home	Alone	Sunny	30	6PM	Carry out & Take away	1d	Male	below21	Single	...	gt8	gt8	
11528	Home	Alone	Sunny	80	6PM	Carry out & Take away	2h	Male	below21	Single	...	gt8	gt8	
11535	Work	Alone	Rainy	55	7AM	Restaurant(<20)	2h	Male	below21	Single	...	gt8	gt8	
10243	Home	Partner	Rainy	55	6PM	Coffee House	1d	Female	below21	Unmarried partner	...	1~3	4~8	

12684 rows x 27 columns

6. SQL:

SELECT age as AGE FROM dataset_1

```
[24]: # In SQL, SELECT age as AGE FROM dataset_1
dt_py.rename(columns={'age':'AGE'},inplace=True)
dt_py
```

	destination	passanger	weather	temperature	time	coupon	expiration	gender	AGE	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaurant2
0	No Urgent Place	Alone	Sunny	55	2PM	Restaurant(<20)	1d	Female	21	Unmarried partner	...	NaN	4~8	
1	No Urgent Place	Friend(s)	Sunny	80	10AM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
2	No Urgent Place	Friend(s)	Sunny	80	10AM	Carry out & Take away	2h	Female	21	Unmarried partner	...	NaN	4~8	
3	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	2h	Female	21	Unmarried partner	...	NaN	4~8	
4	No Urgent Place	Friend(s)	Sunny	80	2PM	Coffee House	1d	Female	21	Unmarried partner	...	NaN	4~8	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
12679	Home	Partner	Rainy	55	6PM	Carry out & Take away	1d	Male	26	Single	...	1~3	4~8	
12680	Work	Alone	Rainy	55	7AM	Carry out & Take away	1d	Male	26	Single	...	1~3	4~8	
12681	Work	Alone	Snowy	30	7AM	Coffee House	1d	Male	26	Single	...	1~3	4~8	
12682	Work	Alone	Snowy	30	7AM	Bar	1d	Male	26	Single	...	1~3	4~8	
12683	Work	Alone	Sunny	80	7AM	Restaurant(20-50)	2h	Male	26	Single	...	1~3	4~8	

12684 rows x 27 columns

Python:

```
dt_py.rename(columns={'age':'AGE'},inplace=True)
```

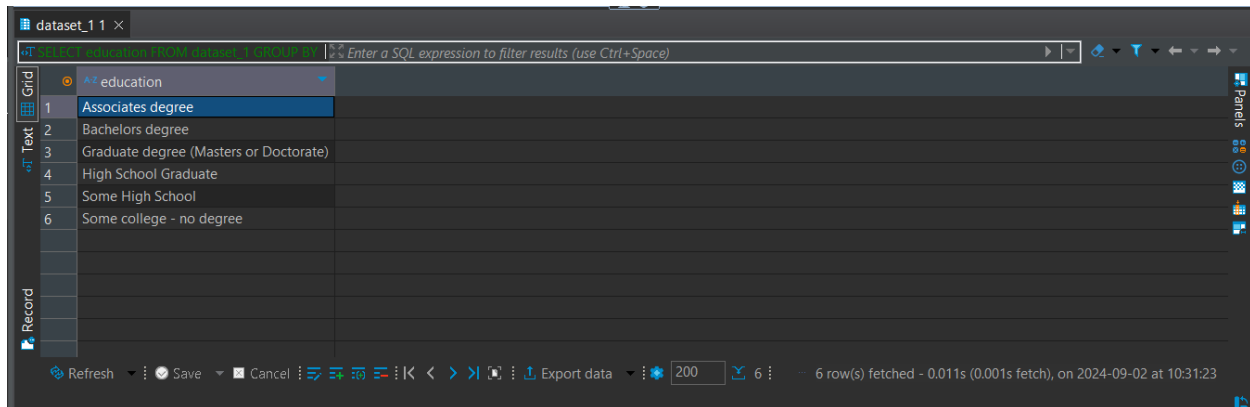
```
dt_py
```

	AGE
1	21
2	21
3	21
4	21
5	21
6	21
7	21
8	21
9	21
10	21
11	21
12	21

200 row(s) fetched - 0.002s (0.001s fetch), on 2024-09-02 at 10:28:10

7. SQL:

SELECT education FROM dataset_1 GROUP BY education



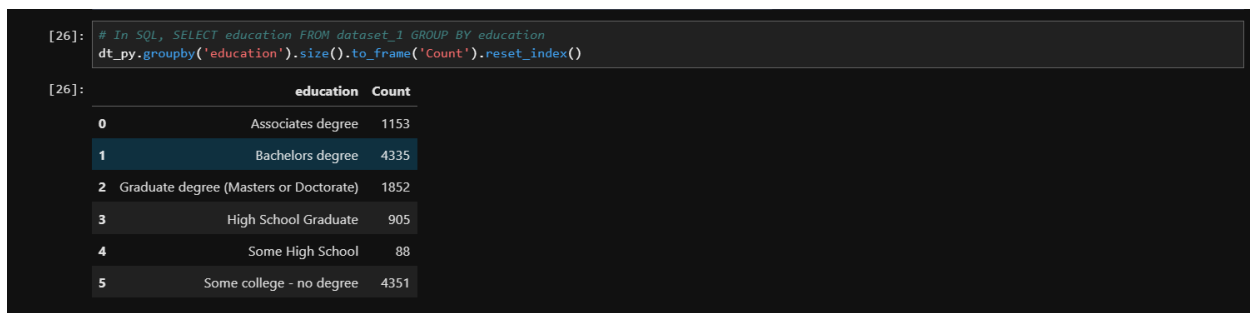
The screenshot shows a SQL query interface with the query `SELECT education FROM dataset_1 GROUP BY education` entered in the top bar. The results are displayed in a table with two columns: 'education' and an empty column for counts. The rows are ordered by the 'education' values.

education	
Associates degree	
Bachelors degree	
Graduate degree (Masters or Doctorate)	
High School Graduate	
Some High School	
Some college - no degree	

At the bottom, a status bar indicates "6 row(s) fetched - 0.011s (0.001s fetch), on 2024-09-02 at 10:31:23".

Python:

```
dt_py.groupby('education').size().to_frame('Count').reset_index()
```



The screenshot shows a Jupyter Notebook cell with the following code:

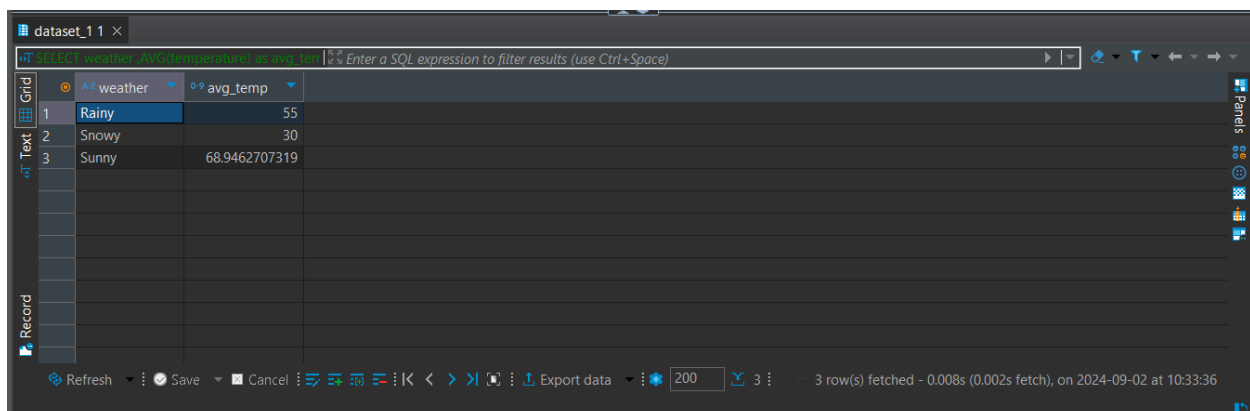
```
[26]: # In SQL, SELECT education FROM dataset_1 GROUP BY education
dt_py.groupby('education').size().to_frame('Count').reset_index()
```

The output of the code is a DataFrame with the following data:

	education	Count
0	Associates degree	1153
1	Bachelors degree	4335
2	Graduate degree (Masters or Doctorate)	1852
3	High School Graduate	905
4	Some High School	88
5	Some college - no degree	4351

8. SQL:

SELECT weather ,AVG(temperature) as avg_temp FROM dataset_1 GROUP BY weather



The screenshot shows a SQL query interface with the query `SELECT weather ,AVG(temperature) as avg_temp FROM dataset_1 GROUP BY weather` entered in the top bar. The results are displayed in a table with two columns: 'weather' and 'avg_temp'. The rows are ordered by the 'weather' values.

weather	avg_temp
Rainy	55
Snowy	30
Sunny	68.9462707319

At the bottom, a status bar indicates "3 row(s) fetched - 0.008s (0.002s fetch), on 2024-09-02 at 10:33:36".

Python:

```
dt_py.groupby('weather')['temperature'].mean().to_frame('avg_temp').reset_index()
```

```
[28]: # In SQL, SELECT weather ,AVG(temperature) as avg_temp FROM dataset_1 GROUP BY weather
dt_py.groupby('weather')['temperature'].mean().to_frame('avg_temp').reset_index()

[28]:
```

	weather	avg_temp
0	Rainy	55.000000
1	Snowy	30.000000
2	Sunny	68.946271

9. SQL:

```
SELECT weather, COUNT(temperature) AS count_temp FROM dataset_1 GROUP BY weather
```

	weather	count_temp
1	Rainy	1,210
2	Snowy	1,405
3	Sunny	10,069

Python:

```
dt_py.groupby('weather')['temperature'].size().to_frame('Count_temp').reset_index()
```

```
[30]: # In SQL, SELECT weather, COUNT(temperature) AS count_temp FROM dataset_1 GROUP BY weather
dt_py.groupby('weather')['temperature'].size().to_frame('Count_temp').reset_index()

[30]:
```

	weather	Count_temp
0	Rainy	1210
1	Snowy	1405
2	Sunny	10069

10. SQL:

```
SELECT weather, COUNT(DISTINCT temperature) AS count_distinct_temp FROM dataset_1 GROUP BY weather
```

dataset_11 x

Enter a SQL expression to filter results (use Ctrl+Space)

	weather	count_distinct_temp
1	Rainy	1
2	Snowy	1
3	Sunny	3

Refresh Save Cancel Export data 200 3 3 row(s) fetched - 0.013s (0.003s fetch), on 2024-09-02 at 12:19:04

Python:

```
[32]: # In SQL, SELECT weather, COUNT(DISTINCT temperature) AS count_distinct_temp FROM dataset_1 GROUP BY weather
dt_py.groupby('weather')['temperature'].nunique().to_frame('count_distinct_temp').reset_index()
```

```
[32]:
```

	weather	count_distinct_temp
0	Rainy	1
1	Snowy	1
2	Sunny	3

11. SQL:

SELECT weather, SUM(temperature) AS sum_temp FROM dataset_1 GROUP BY weather

dataset_11 x

Enter a SQL expression to filter results (use Ctrl+Space)

	weather	sum_temp
1	Rainy	66,550
2	Snowy	42,150
3	Sunny	694,220

Refresh Save Cancel Export data 200 3 3 row(s) fetched - 0.007s (0.001s fetch), on 2024-09-02 at 12:21:50

Python:

dt_py.groupby('weather')['temperature'].sum().to_frame('sum_temp').reset_index()

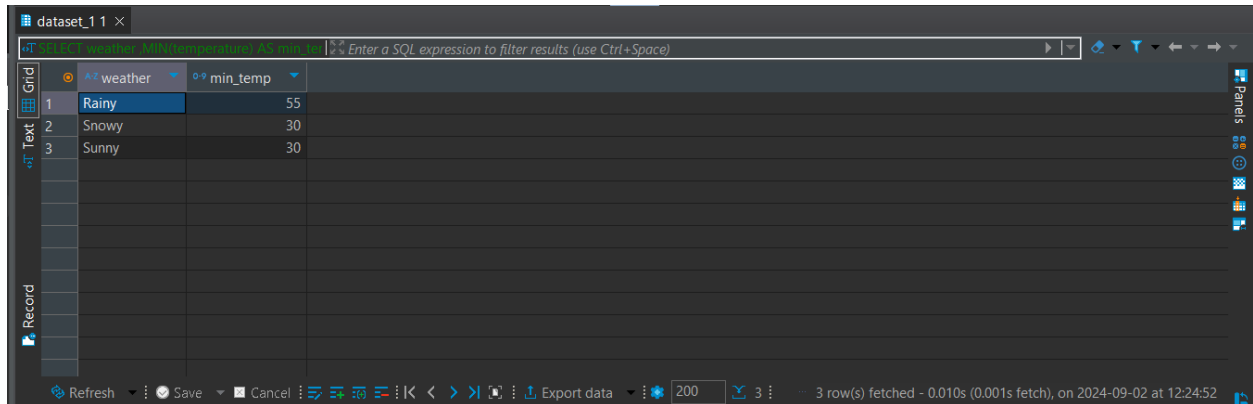
```
[34]: # In SQL, SELECT weather, SUM(temperature) AS sum_temp FROM dataset_1 GROUP BY weather
dt_py.groupby('weather')['temperature'].sum().to_frame('sum_temp').reset_index()
```

```
[34]:
```

	weather	sum_temp
0	Rainy	66550
1	Snowy	42150
2	Sunny	694220

12. SQL:

SELECT weather ,MIN(temperature) AS min_temp FROM dataset_1 GROUP BY weather



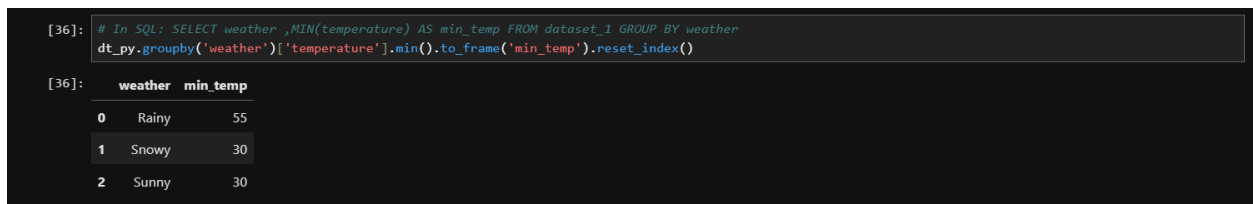
The screenshot shows a SQL query interface with the query: `SELECT weather ,MIN(temperature) AS min_temp FROM dataset_1 GROUP BY weather`. The results are displayed in a table with columns 'weather' and 'min_temp'. The data is as follows:

weather	min_temp
Rainy	55
Snowy	30
Sunny	30

The interface also shows a status bar at the bottom indicating '3 row(s) fetched - 0.010s (0.001s fetch), on 2024-09-02 at 12:24:52'.

Python:

```
dt_py.groupby('weather')['temperature'].min().to_frame('min_temp').reset_index()
```



The screenshot shows a Python Jupyter Notebook with the following code and output:

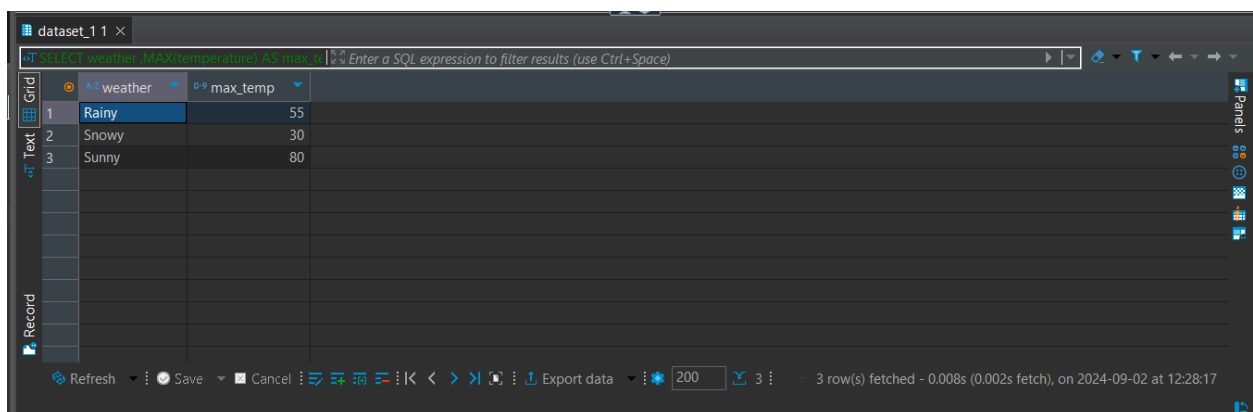
```
[36]: # In SQL: SELECT weather ,MIN(temperature) AS min_temp FROM dataset_1 GROUP BY weather
dt_py.groupby('weather')['temperature'].min().to_frame('min_temp').reset_index()
```

The output is a DataFrame with columns 'weather' and 'min_temp'.

	weather	min_temp
0	Rainy	55
1	Snowy	30
2	Sunny	30

13. SQL:

SELECT weather ,MAX(temperature) AS max_temp FROM dataset_1 GROUP BY weather



The screenshot shows a SQL query interface with the query: `SELECT weather ,MAX(temperature) AS max_temp FROM dataset_1 GROUP BY weather`. The results are displayed in a table with columns 'weather' and 'max_temp'. The data is as follows:

weather	max_temp
Rainy	55
Snowy	30
Sunny	80

The interface also shows a status bar at the bottom indicating '3 row(s) fetched - 0.008s (0.002s fetch), on 2024-09-02 at 12:28:17'.

Python:

```
dt_py.groupby('weather')['temperature'].max().to_frame('max_temp').reset_index()
```

```
[38]: # In SQL: SELECT weather ,MAX(temperature) AS max_temp FROM dataset_1 GROUP BY weather
dt_py.groupby('weather')['temperature'].max().to_frame('max_temp').reset_index()
```

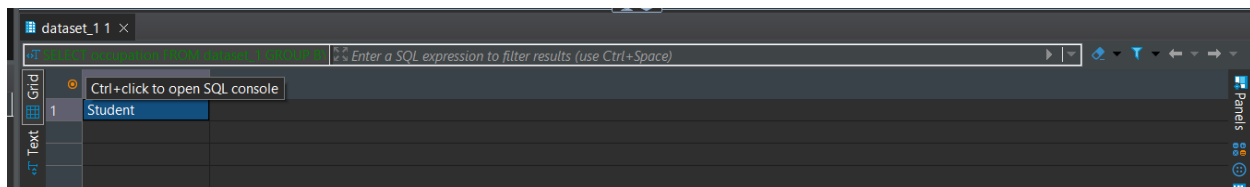
```
[38]:
```

	weather	max temp
0	Rainy	55
1	Snowy	30
2	Sunny	80

Author: www.linkedin.com/in/shabanaaalina14

14. SQL:

```
SELECT OCCUPATION FROM dataset_1 GROUP BY OCCUPATION HAVING
occupation='Student'
```



The screenshot shows a SQL query interface with a table named 'dataset_1'. The query is: `SELECT OCCUPATION FROM dataset_1 GROUP BY OCCUPATION HAVING occupation='Student'`. The result is displayed in a grid view with one row and one column, showing the value 'Student'.

	OCCUPATION
1	Student

Python:

```
dt_py.groupby('occupation').filter(lambda x: x['occupation'].iloc[0]
=='Student').groupby('occupation').size()
```

```
•[42]: # In SQL: SELECT OCCUPATION FROM dataset_1 GROUP BY OCCUPATION HAVING occupation='Student'
dt_py.groupby('OCCUPATION').filter(lambda x: x['OCCUPATION'].iloc[0] == 'Student').groupby('OCCUPATION').size()
```

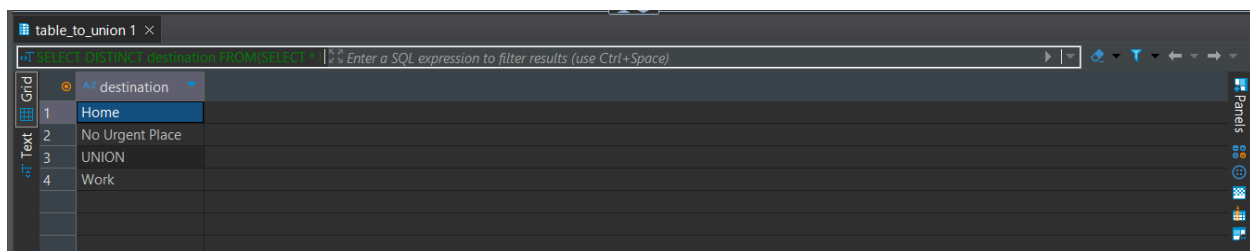
```
[42]:
```

	OCCUPATION	size
Student	Student	1584

dtype: int64

15. SQL:

```
SELECT DISTINCT destination FROM(SELECT * FROM dataset_1 UNION SELECT *
FROM table_to_union)
```



The screenshot shows a SQL query interface with a table named 'table_to_union'. The query is: `SELECT DISTINCT destination FROM(SELECT * FROM dataset_1 UNION SELECT * FROM table_to_union)`. The result is displayed in a grid view with four rows and one column, showing the values 'Home', 'No Urgent Place', 'UNION', and 'Work'.

	destination
1	Home
2	No Urgent Place
3	UNION
4	Work

Python:

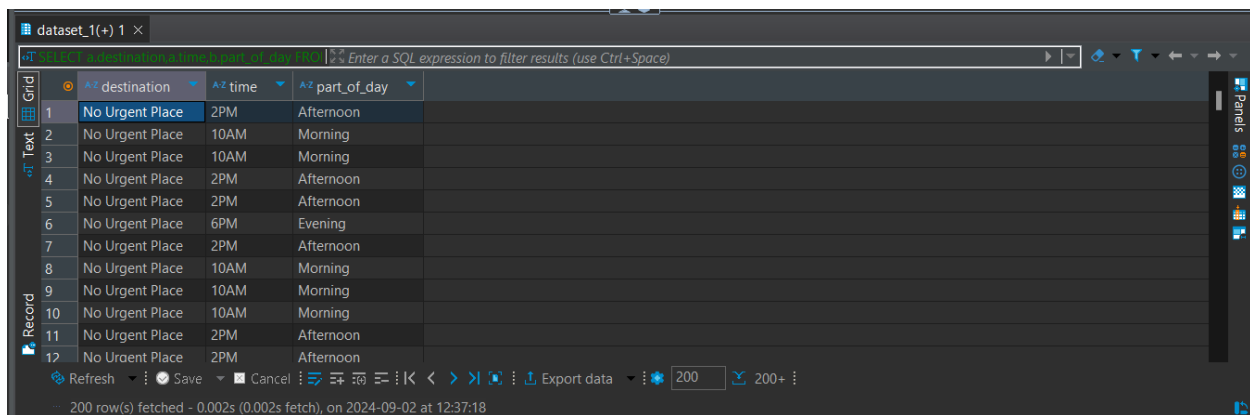
```
pd.concat([dt_py, dt_py])['destination'].drop_duplicates()
```

```
[50]: # In SQL: SELECT DISTINCT destination FROM(SELECT * FROM dataset_1 UNION SELECT * FROM table_to_union)
pd.concat([dt_py, dt_py])['destination'].drop_duplicates()

[50]: 0    No Urgent Place
      13           Home
      16           Work
      Name: destination, dtype: object
```

16. SQL:

```
SELECT destination ,passenger FROM(SELECT*FROM dataset_1 WHERE passenger = 'Alone')
```



	destination	time	part_of_day
1	No Urgent Place	2PM	Afternoon
2	No Urgent Place	10AM	Morning
3	No Urgent Place	10AM	Morning
4	No Urgent Place	2PM	Afternoon
5	No Urgent Place	2PM	Afternoon
6	No Urgent Place	6PM	Evening
7	No Urgent Place	2PM	Afternoon
8	No Urgent Place	10AM	Morning
9	No Urgent Place	10AM	Morning
10	No Urgent Place	10AM	Morning
11	No Urgent Place	2PM	Afternoon
12	No Urgent Place	2PM	Afternoon

Python:

```
dt_py[dt_py['passanger'] == 'Alone'][['destination', 'passanger']]
```

```
[76]: # In SQL: SELECT destination ,passenger FROM(SELECT*FROM dataset_1 WHERE passenger = 'Alone')
dt_py[dt_py['passanger'] == 'Alone'][['destination', 'passanger']]

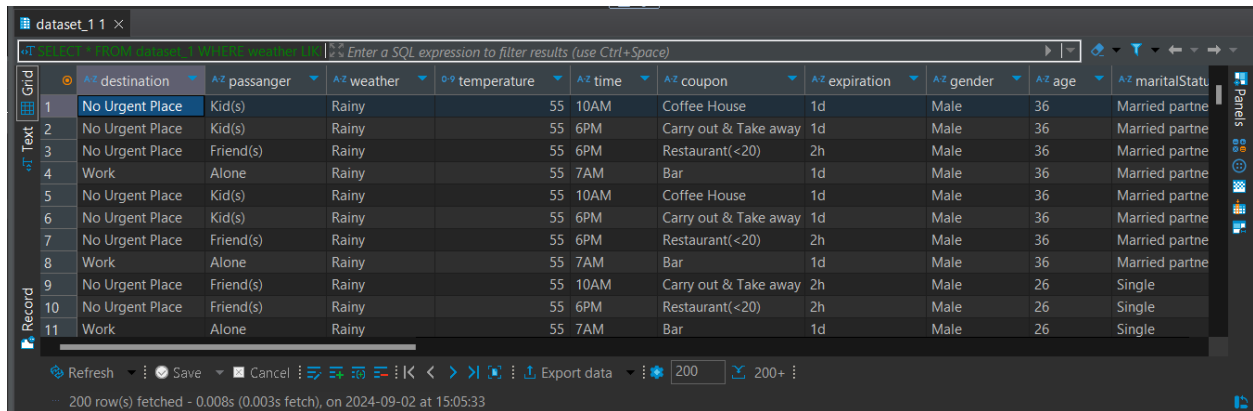
[76]:
```

	destination	passanger
0	No Urgent Place	Alone
13	Home	Alone
14	Home	Alone
15	Home	Alone
16	Work	Alone
...	...	...
12676	Home	Alone
12680	Work	Alone
12681	Work	Alone
12682	Work	Alone
12683	Work	Alone

7305 rows × 2 columns

17. SQL:

SELECT * FROM dataset_1 WHERE weather LIKE 'Rainy'

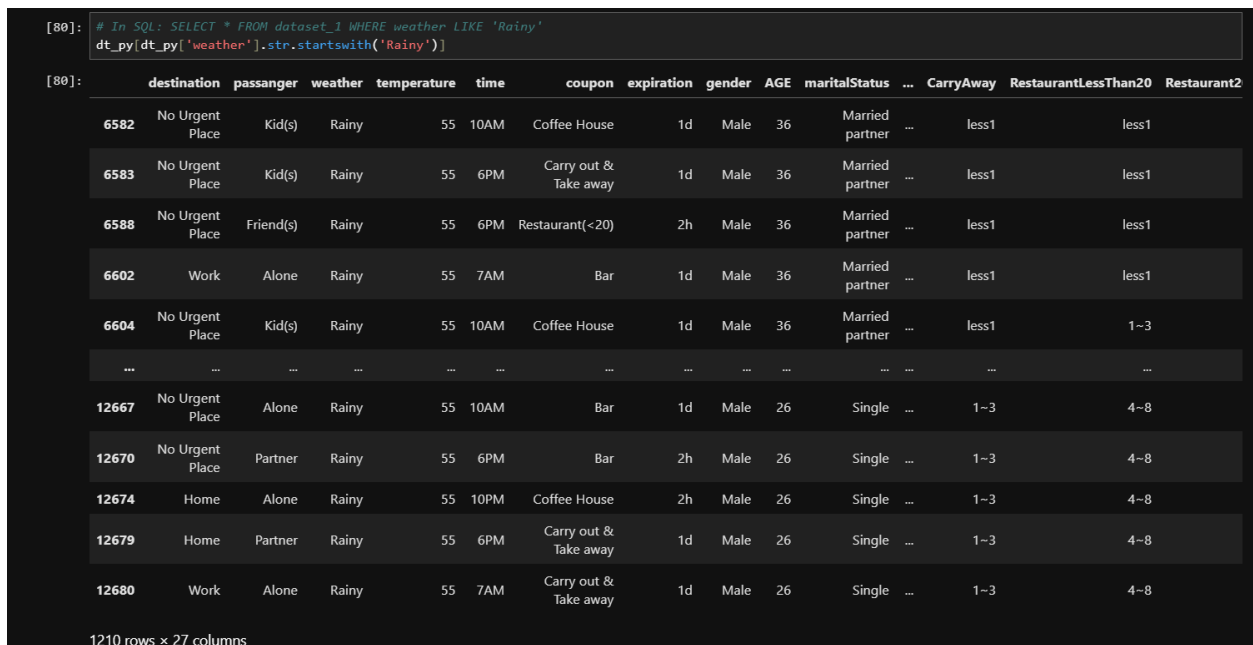


The screenshot shows a SQL query interface with a table named 'dataset_1'. The query is 'SELECT * FROM dataset_1 WHERE weather LIKE 'Rainy''. The table has 11 rows of data. The columns are: destination, passanger, weather, temperature, time, coupon, expiration, gender, age, and maritalStatu. The data is as follows:

	destination	passanger	weather	temperature	time	coupon	expiration	gender	age	maritalStatu
1	No Urgent Place	Kid(s)	Rainy	55	10AM	Coffee House	1d	Male	36	Married partne
2	No Urgent Place	Kid(s)	Rainy	55	6PM	Carry out & Take away	1d	Male	36	Married partne
3	No Urgent Place	Friend(s)	Rainy	55	6PM	Restaurant(<20)	2h	Male	36	Married partne
4	Work	Alone	Rainy	55	7AM	Bar	1d	Male	36	Married partne
5	No Urgent Place	Kid(s)	Rainy	55	10AM	Coffee House	1d	Male	36	Married partne
6	No Urgent Place	Kid(s)	Rainy	55	6PM	Carry out & Take away	1d	Male	36	Married partne
7	No Urgent Place	Friend(s)	Rainy	55	6PM	Restaurant(<20)	2h	Male	36	Married partne
8	Work	Alone	Rainy	55	7AM	Bar	1d	Male	36	Married partne
9	No Urgent Place	Friend(s)	Rainy	55	10AM	Carry out & Take away	2h	Male	26	Single
10	No Urgent Place	Friend(s)	Rainy	55	6PM	Restaurant(<20)	2h	Male	26	Single
11	Work	Alone	Rainy	55	7AM	Bar	1d	Male	26	Single

Python:

```
dt_py[dt_py['weather'].str.startswith('Rainy')]
```



The screenshot shows a Python Jupyter Notebook with a cell containing the following code:

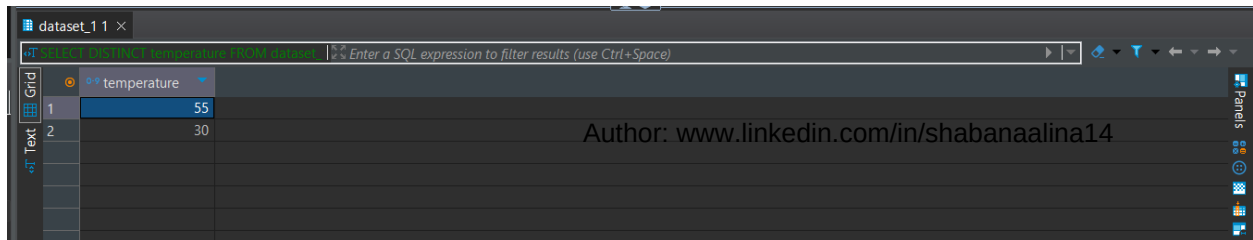
```
[80]: # In SQL: SELECT * FROM dataset_1 WHERE weather LIKE 'Rainy'
dt_py[dt_py['weather'].str.startswith('Rainy')]
```

The output of the code is a table with 1210 rows and 27 columns. The columns are: destination, passanger, weather, temperature, time, coupon, expiration, gender, AGE, maritalStatus, ..., CarryAway, RestaurantLessThan20, Restaurant2. The data is as follows:

	destination	passanger	weather	temperature	time	coupon	expiration	gender	AGE	maritalStatus	...	CarryAway	RestaurantLessThan20	Restaurant2
6582	No Urgent Place	Kid(s)	Rainy	55	10AM	Coffee House	1d	Male	36	Married partner	...	less1	less1	
6583	No Urgent Place	Kid(s)	Rainy	55	6PM	Carry out & Take away	1d	Male	36	Married partner	...	less1	less1	
6588	No Urgent Place	Friend(s)	Rainy	55	6PM	Restaurant(<20)	2h	Male	36	Married partner	...	less1	less1	
6602	Work	Alone	Rainy	55	7AM	Bar	1d	Male	36	Married partner	...	less1	less1	
6604	No Urgent Place	Kid(s)	Rainy	55	10AM	Coffee House	1d	Male	36	Married partner	...	less1	1-3	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
12667	No Urgent Place	Alone	Rainy	55	10AM	Bar	1d	Male	26	Single	...	1-3	4-8	
12670	No Urgent Place	Partner	Rainy	55	6PM	Bar	2h	Male	26	Single	...	1-3	4-8	
12674	Home	Alone	Rainy	55	10PM	Coffee House	2h	Male	26	Single	...	1-3	4-8	
12679	Home	Partner	Rainy	55	6PM	Carry out & Take away	1d	Male	26	Single	...	1-3	4-8	
12680	Work	Alone	Rainy	55	7AM	Carry out & Take away	1d	Male	26	Single	...	1-3	4-8	

18. SQL:

SELECT DISTINCT temperature FROM dataset_1 WHERE temperature BETWEEN 30 AND 75

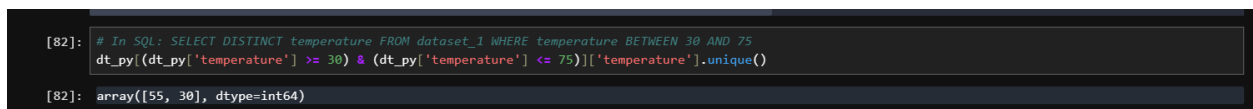


The screenshot shows a data visualization tool interface. At the top, a SQL query is entered: `SELECT DISTINCT temperature FROM dataset_1 WHERE temperature BETWEEN 30 AND 75`. Below the query, a table displays the results. The table has two columns: an index and 'temperature'. The first row shows the value 55, and the second row shows the value 30. A watermark 'Author: www.linkedin.com/in/shabanaalina14' is visible across the middle of the table.

	temperature
1	55
2	30

Python:

```
dt_py[(dt_py['temperature'] >= 30) & (dt_py['temperature'] <= 75)][['temperature']].unique()
```



The screenshot shows a Jupyter Notebook cell with the following code and output:

```
[82]: # In SQL: SELECT DISTINCT temperature FROM dataset_1 WHERE temperature BETWEEN 30 AND 75
      dt_py[(dt_py['temperature'] >= 30) & (dt_py['temperature'] <= 75)][['temperature']].unique()

[82]: array([55, 30], dtype=int64)
```